

Lecture 13

Intro to Error Handling

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



Error Handling Challenges

- An abstraction that may run into an error:
 - Is **obscure** if the error case is not obvious or easy to forget.
 - May be more **complex** even when we do address the error case.
- Partial abstractions introduce coupling:
 - All dependents are also partial, unless somehow insulated.
- Encapsulation boundaries can complicate error handling.



Approaches to Error Handling

- Exceptions
- “Errors as Values”
- Undefined Behavior

Exceptions

- An alternate control flow path vs. return;
- Relieves the function of the obligation to produce a value.
- Used when “I can’t do my job anymore”.

```
function divide(a: number, b: number): number {  
  if (b === 0) {  
    throw new Error("Cannot divide by zero");  
  }  
  return a / b;  
}  
  
async function divideExample() {  
  const a = await prompt_num("Enter numerator:");  
  const b = await prompt_num("Enter denominator:");  
  
  try {  
    const result = divide(a, b);  
  
    // skipped if error thrown  
    console.log("Result:", result);  
  }  
  catch (e) {  
    console.log("Division error!")  
  }  
}
```

Typescript

Typed Exceptions

- We may use custom exception types to carry information about what happened.
- A catch block specifies what it can handle.
- Note the rethrow for unhandled errors. We essentially opt-in to the types we handle.

```
class FileError extends Error { ... }
class FileNotFoundError extends FileError { ... }
class PermissionError extends FileError { ... }
class ParseError extends FileError { ... }

// Assume this function may throw above errors
declare function readFileOrThrow(path: string): FileData;

try { // Example usage
  const file = readFileOrThrow(path);
  console.log(`Read ${file.name}: ${file.content}`);
} catch (e) {
  if (e instanceof FileNotFoundError) {
    console.log(`Not found: ${e.path}`);
  } else if (e instanceof PermissionError) {
    console.log(`No access: ${e.path}`);
  } else if (e instanceof ParseError) {
    console.log(`Bad file: ${e.message}`);
  } else {
    throw e; // re-throw unhandled errors
  }
}
```

Typescript



Checked Exceptions

- Functions must declare the types they may throw.
- The compiler verifies that catch every possible thrown type (unless the calling function also declares it may throw those).

```
class FileNotFoundException extends Exception { ... }
class PermissionException extends FileNotFoundException { ... }
class ParseException extends FileNotFoundException { ... }

public static
FileData readFile(String path) throws FileNotFoundException { ... }

public static void exampleUsage(String path) {
    try {
        FileData file = readFile(path);
        println("Read " + file.name() + ": " + file.content());
    } catch (FileNotFoundException e) {
        println("Not found: " + e.getPath());
    } catch (PermissionException e) {
        println("No access: " + e.getPath());
    } catch (ParseException e) {
        println("Parse error: " + e.getMessage());
    } catch (FileNotFoundException e) {
        println("Bad file: " + e.getMessage());
    }
}
```

Java



Errors as Values

- No separate control flow path for errors.
- Functions don't bail out, instead they just return an alternate result.

```
function divide(a: number, b: number): number | "Error" {  
  if (b === 0) {  
    return "Error";  
  } else {  
    return a / b;  
  }  
}  
  
const result2 = divide(10, 0);  
if (result2 !== "Error") {  
  console.log("Result:", result2);  
} else {  
  console.error("Error: Cannot divide by zero");  
}
```

Typescript



Errors as Values

- Often use a standard “Result Type” pattern for success vs. errors.
- We can’t access the result unless we check the `.ok` member.
- Type narrowing in either branch allows `.value` or `.error` (but not both).

```
type Result<T, E> =  
  { ok: true; value: T } | { ok: false; error: E };  
  
function Ok<T>(v: T): Result<T, never> {  
  return { ok: true, value: v };  
};  
  
function Err<E>(e: E): Result<never, E> {  
  return { ok: false, error: e };  
};  
  
function div(a: number, b: number): Result<number, string> {  
  if (b === 0) {  
    return Err("Cannot divide by zero");  
  } else {  
    return Ok(a / b);  
  }  
}  
  
const result1 = divide(10, 0);  
if (result1.ok) {  
  console.log("Result:", result1.value);  
} else {  
  console.error("Error:", result1.error);  
}
```

**Typescript (Rust-style
Result type)**



Undefined Behavior

- Specify preconditions for the function to work correctly.
- e.g. REQUIRES clause
- Not my problem if you break the rules!

```
// REQUIRES: b is nonzero
int divide(int a, int b) {
    if (b != 0) {
        return a / b;
    }
    else {
        return 42; // ehhh whatever
    }
}
```

```
// REQUIRES: b is nonzero
int divide(int a, int b) {
    assert(b != 0); // better: crash out
    return a / b;
}
```

C++

LBYL vs. EAFP Philosophies

- Assume some operation that might fail...

```
const a = await prompt_num("Enter numerator:");
const b = await prompt_num("Enter denominator:");
// Let's divide these!
```

LBYL: Look Before You Leap

```
if (b === 0) {
  console.log("Cannot divide by zero");
}
else {
  console.log("Result:", divide(a, b));
}
```

EAFP: Easier to Ask for Forgiveness than Permission

```
try {
  const result = divide(a, b);
  console.log("Result:", result);
}
catch (e) {
  console.log("Division error!")
}
```

(EAFP pattern can also be used with “errors as values” via checking the a result type.)



Taxonomy of Exceptionality

- **Fatal**

An exception is thrown because of a fatal system error (e.g. out of memory)

- **Preventable**

An exception is thrown because of a programming error that could be fixed.

- **Exogenous**

An exception is thrown because an operation/resource out of our control fails.

- **Vexing**

An exception is thrown because of poor design, wrong encapsulations, etc.



Break Time

bit.ly/4sMbzw9

